

Tarefa 11: Particionamento de Dados e Balanceamento de Carga: Difusão Viscosa 2D com OpenMP — schedule e collapse

Professor: Samuel Xavier de Souza
Aluno: Vinícius Silva do Carmo
Junho de 2026

1. Objetivo

Simular a difusão viscosa de um fluido (Navier-Stokes simplificado) em grade 2D usando diferenças finitas, validar a estabilidade numérica do esquema explícito e paralelizar o stencil com OpenMP, comparando o impacto das cláusulas schedule (static, dynamic, guided) e collapse no desempenho.

2. Ambiente de Execução

Item	Valor
CPU	AMD Ryzen 5 5600G (4 núcleos)
Grade	512 × 512 células
Passos de tempo (benchmark)	1000
Compilador	GCC -O2 -fopenmp
Sistema Operacional	Linux (WSL2)

3. Modelo Numérico

Desconsiderando pressão e forças externas, a equação de Navier-Stokes se reduz à equação de difusão $\partial u / \partial t = \nu \nabla^2 u$, discretizada por diferenças finitas explícitas (Euler Forward) com um stencil de 5 pontos. $r = \nu \cdot \Delta t / \Delta x^2$ é o parâmetro de estabilidade; em 2D o esquema exige $r \leq 0,25$. Com $\nu = 0,1$, $\Delta x = 1,0$ e $\Delta t = 0,1$, obtém-se $r = 0,01$, bem abaixo do limite — o esquema é incondicionalmente estável para esses parâmetros.

$$u[i][j]^{(n+1)} = u[i][j]^{(n)} + r * (u[i+1][j] + u[i-1][j] + u[i][j+1] + u[i][j-1] - 4*u[i][j])$$

4. Implementação Paralela

O stencil é aplicado a cada célula interior em cada passo de tempo. Como cada célula depende apenas do passo anterior, o loop espacial é paralelizável diretamente — cada linha (ou célula, com collapse) é independente das demais dentro do mesmo passo.

4.1 schedule(static)

Divide as 510 linhas do loop externo em blocos contíguos de tamanho igual entre as 4 threads. Acesso row-major favorece o cache; overhead de coordenação mínimo.

```
#pragma omp parallel for num_threads(4) schedule(static)
```

```

for (int i = 1; i < NX-1; i++)
    for (int j = 1; j < NY-1; j++)
        u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
                                   + u[i][j+1] + u[i][j-1]
                                   - 4.0*u[i][j]);

```

4.2 collapse(2) + static

Funde os dois loops em um único espaço de 260.100 iterações, permitindo divisão mais fina entre as threads ao custo de recalculer os índices 2D a partir do índice linear colapsado.

```

#pragma omp parallel for num_threads(4) schedule(static) collapse(2)
for (int i = 1; i < NX-1; i++)
    for (int j = 1; j < NY-1; j++)
        u_new[i][j] = /* mesmo cálculo */;

```

Além de static e collapse(2), também foram testadas as variantes static com chunk explícito (NX/4), dynamic (chunk=16) e guided, todas aplicadas ao mesmo loop externo.

5. Validação

5.1 Teste A — Campo uniforme

Interior inicializado em $u = 1,0$ com bordas fixas em 0 (Dirichlet). O quarto central da grade, distante das bordas, permanece exatamente em 1,0 ao longo de toda a simulação ($\max|u - 1,0| = 0,00e+00$), confirmando que nenhuma instabilidade artificial é introduzida pelo esquema.

5.2 Teste B — Perturbação gaussiana

Campo nulo com uma gaussiana de amplitude 10 e $\sigma = 20$ células no centro. A energia total decai suavemente de $1,257e+05$ para $1,244e+05$ ao longo de 200 passos, consistente com a difusão física e a absorção de energia nas bordas.

Difusao viscosa - perturbacao gaussiana (instantaneo)

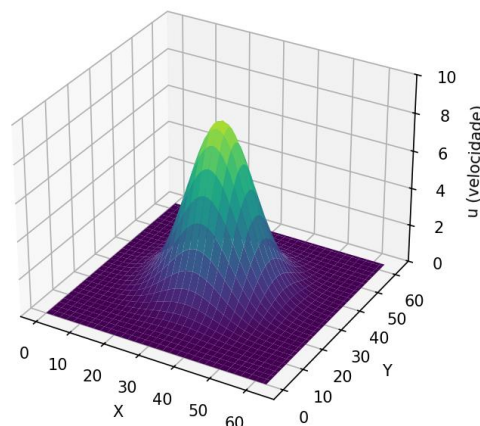


Figura 1 — Instantâneo da perturbação gaussiana após parte da evolução temporal.

6. Resultados

Grade 512×512, 1000 passos de tempo, 4 threads:

Versão	Tempo (s)	Speedup
Sequencial	0,298	1,00
static	0,240	1,24
static, chunk=NX/4	0,187	1,59
dynamic, chunk=16	0,171	1,74
guided	0,165	1,81
collapse(2) + static	0,174	1,71

7. Análise

static divide as 510 linhas em 4 blocos contíguos; o acesso row-major favorece o cache e o overhead de coordenação é mínimo, mas o speedup fica em apenas 1,24× porque o GCC com -O2 já otimiza fortemente o kernel sequencial. Reduzir o chunk (static com chunk=NX/4) e, principalmente, usar escalonamento dinâmico (dynamic, guided) melhora o resultado: o guided alcança o melhor speedup (1,81×) por iniciar com blocos grandes e reduzi-los progressivamente, equilibrando melhor a carga residual entre as threads ao final do loop.

O collapse(2) permite divisão mais fina, mas o custo de recalcular os índices 2D a partir do índice linear e a menor localidade de cache por thread reduzem o ganho (1,71×), ficando abaixo do guided. Essa técnica tende a ser mais vantajosa quando o loop externo tem poucas iterações em relação ao número de threads — o que não é o caso aqui, já que NX = 512 é grande.

8. Conclusão

Cláusula	Speedup	Quando usar
static	1,24×	Carga uniforme, loops muito rápidos
static, chunk	1,59×	Carga uniforme, chunk ajustado ao cache
dynamic	1,74×	Carga moderadamente variável
guided	1,81×	Melhor para este stencil
collapse(2)+static	1,71×	NX pequeno, poucas linhas por thread

Para stencils 2D com carga uniforme e grade grande, guided e dynamic superam o static simples porque o kernel sequencial já está bem otimizado pelo compilador, e o ganho do paralelismo vem principalmente do melhor uso do cache via escalonamento fino — não da divisão bruta do trabalho. A simulação confirmou ainda que o esquema explícito de diferenças finitas é numericamente estável para $r = 0,01$, muito abaixo do limite de 0,25.

9. Código Fonte

```

1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <string.h>
6  #include <time.h>
7  #include <omp.h>
8
9  /* — Parâmetros da simulação — */
10 #define NX      512 /* células na direção x */
11 #define NY      512 /* células na direção y */
12 #define NT      1000 /* passos de tempo para benchmark */
13 #define NT_VAL   200 /* passos para validação */
14 #define NU      0.1 /* viscosidade cinemática */
15 #define DX      1.0 /* espaçamento espacial */
16 #define DT      0.1 /* passo de tempo */
17 #define NTHREADS 4
18
19 /*  $r = \nu \Delta t / \Delta x^2 = 0.1 \times 0.1 / 1.0 = 0.01 \leq 0.25 \rightarrow$  estável */
20 #define R (NU * DT / (DX * DX))
21
22 #define BILLION 1000000000.0
23
24 static double elapsed(struct timespec a, struct timespec b) {
25     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
26 }
27
28 /* — Utilidades de grade — */
29
30 static double (*alloc_grid(void))[NY] {
31     return calloc(NX, NY * sizeof(double));
32 }
33
34 static void copy_grid(double dst[NX][NY], const double src[NX][NY]) {
35     memcpy(dst, src, NX * NY * sizeof(double));
36 }
37
38 /* Energia total:  $\sum u[i][j]^2$  */
39 static double energia(const double u[NX][NY]) {
40     double e = 0.0;
41     for (int i = 0; i < NX; i++)
42         for (int j = 0; j < NY; j++)
43             e += u[i][j] * u[i][j];
44     return e;
45 }
46
47 /* Desvio máximo no quarto central da grade (longe das bordas Dirichlet) */
48 static double max_delta_centro(const double u[NX][NY], double esperado) {
49     double d = 0.0;
50     int i0 = NX/4, i1 = 3*NX/4;
51     int j0 = NY/4, j1 = 3*NY/4;
52     for (int i = i0; i < i1; i++)
53         for (int j = j0; j < j1; j++) {
54             double diff = fabs(u[i][j] - esperado);
55             if (diff > d) d = diff;
56         }
57     return d;
58 }
59
60 /* Perturbação gaussiana centrada na grade */
61 static void gaussiana(double u[NX][NY], double amplitude, double sigma) {
62     int ci = NX / 2, cj = NY / 2;
63     for (int i = 0; i < NX; i++)
64         for (int j = 0; j < NY; j++) {
65             double di = i - ci, dj = j - cj;
66             u[i][j] += amplitude * exp(-(di*di + dj*dj) / (2.0 * sigma * sigma));
67         }
68 }
69

```

```

69
70 /* — Stencil de difusão: um passo de tempo ————— */
71 /* Condição de contorno: Dirichlet u=0 nas bordas (implícito: não atualiza bordas). */
72
73 /* Versão sequencial */
74 static void passo_seq(const double u[NX][NY], double u_new[NX][NY]) {
75     for (int i = 1; i < NX-1; i++)
76         for (int j = 1; j < NY-1; j++)
77             u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
78                                     + u[i][j+1] + u[i][j-1]
79                                     - 4.0*u[i][j]);
80 }
81
82 /* schedule(static) */
83 static void passo_static(const double u[NX][NY], double u_new[NX][NY]) {
84     #pragma omp parallel for num_threads(NTHREADS) schedule(static)
85     for (int i = 1; i < NX-1; i++)
86         for (int j = 1; j < NY-1; j++)
87             u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
88                                     + u[i][j+1] + u[i][j-1]
89                                     - 4.0*u[i][j]);
90 }
91
92 /* schedule(static, chunk) – chunk explícito */
93 static void passo_static_chunk(const double u[NX][NY], double u_new[NX][NY]) {
94     #pragma omp parallel for num_threads(NTHREADS) schedule(static, NX/NTHREADS)
95     for (int i = 1; i < NX-1; i++)
96         for (int j = 1; j < NY-1; j++)
97             u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
98                                     + u[i][j+1] + u[i][j-1]
99                                     - 4.0*u[i][j]);
100 }
101
102 /* schedule(dynamic, 16) */
103 static void passo_dynamic(const double u[NX][NY], double u_new[NX][NY]) {
104     #pragma omp parallel for num_threads(NTHREADS) schedule(dynamic, 16)
105     for (int i = 1; i < NX-1; i++)
106         for (int j = 1; j < NY-1; j++)
107             u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
108                                     + u[i][j+1] + u[i][j-1]
109                                     - 4.0*u[i][j]);
110 }
111
112 /* schedule(guided) */
113 static void passo_guided(const double u[NX][NY], double u_new[NX][NY]) {
114     #pragma omp parallel for num_threads(NTHREADS) schedule(guided)
115     for (int i = 1; i < NX-1; i++)
116         for (int j = 1; j < NY-1; j++)
117             u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
118                                     + u[i][j+1] + u[i][j-1]
119                                     - 4.0*u[i][j]);
120 }
121
122 /* collapse(2) + static – colapsa os dois loops em um espaço de iterações único */
123 static void passo_collapse(const double u[NX][NY], double u_new[NX][NY]) {
124     #pragma omp parallel for num_threads(NTHREADS) schedule(static) collapse(2)
125     for (int i = 1; i < NX-1; i++)
126         for (int j = 1; j < NY-1; j++)
127             u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
128                                     + u[i][j+1] + u[i][j-1]
129                                     - 4.0*u[i][j]);
130 }
131

```

```

132 /* — Evolução temporal genérica — */
133
134 typedef void (*passo_fn)(const double[NX][NY], double[NX][NY]);
135
136 static double evoluir(double u[NX][NY], double u_new[NX][NY],
137                       passo_fn fn, int nt) {
138     struct timespec ini, fim;
139     clock_gettime(CLOCK_MONOTONIC, &ini);
140     for (int t = 0; t < nt; t++) {
141         fn(u, u_new);
142         double (*tmp)[NY] = u;
143         /* troca ponteiros — não permitido com arrays estáticos; copiar */
144         memcpy((void *)u, u_new, NX * NY * sizeof(double));
145         (void)tmp;
146     }
147     clock_gettime(CLOCK_MONOTONIC, &fim);
148     return elapsed(ini, fim);
149 }
150
151 /* — main — */
152
153 int main(void) {
154     double (*u)[NY] = alloc_grid();
155     double (*u_new)[NY] = alloc_grid();
156     double (*base)[NY] = alloc_grid();
157
158     printf("Simulação de difusão viscosa (Navier-Stokes simplificado)\n");
159     printf("Grade: %dx%d | r = v·Δt/Δx² = %.4f (limite estabilidade: 0.25)\n\n",
160           NX, NY, R);
161
162     /* — Fase 1: validação — */
163     printf("=== Fase 1: Validação (%d passos) ===\n\n", NT_VAL);
164
165     /* Teste A: campo uniforme — bordas fixas em 0 drenam a solução lentamente;
166        mas o interior longe das bordas deve permanecer muito próximo de 1.0 */
167     for (int i = 1; i < NX-1; i++)
168         for (int j = 1; j < NY-1; j++)
169             u[i][j] = 1.0;
170     double e_ini = energia(u);
171     evoluir(u, u_new, passo_seq, NT_VAL);
172     printf("Teste A — campo uniforme (interior):\n");
173     printf(" Energia inicial: %.6e\n", e_ini);
174     printf(" Energia final: %.6e\n", energia(u));
175     printf(" max|u[i][j] - 1.0| no quarto central: %.2e (bordas Dirichlet=0 afetam apenas a periferia)\n\n",
176           max_delta_centro(u, 1.0));
177
178     /* Teste B: perturbação gaussiana sobre campo nulo */
179     memset(u, 0, NX * NY * sizeof(double));
180     gaussiana(u, 10.0, 20.0);
181     e_ini = energia(u);
182     copy_grid(base, u); /* salva estado inicial */
183     evoluir(u, u_new, passo_seq, NT_VAL);
184     printf("Teste B — perturbação gaussiana (difusão suave):\n");
185     printf(" Energia inicial: %.6e\n", e_ini);
186     printf(" Energia final: %.6e (decaiu por difusão + bordas)\n\n",
187           energia(u));
188
189     /* — Fase 2: benchmark schedule/collapse — */
190     printf("=== Fase 2: Desempenho (%dx%d, %d passos, %d threads) ===\n\n",
191           NX, NY, NT, NTHREADS);
192     printf("%-28s %-10s %-8s\n", "Versão", "Tempo(s)", "Speedup");
193     printf("%-28s %-10s %-8s\n",
194           "-----", "-----", "-----");
195
196     struct { const char *nome; passo_fn fn; } versoes[] = {
197         { "Sequencial", passo_seq },
198         { "static", passo_static },
199         { "static,chunk=NX/4", passo_static_chunk },
200         { "dynamic,chunk=16", passo_dynamic },
201         { "guided", passo_guided },
202         { "collapse(2)+static", passo_collapse },
203     };
204     int nv = sizeof(versoes) / sizeof(versoes[0]);
205     double t_base = 0.0;

```

```

206
207     for (int v = 0; v < nv; v++) {
208         /* reinicia com perturbação gaussiana a cada versão */
209         memset(u, 0, NX * NY * sizeof(double));
210         gaussiana(u, 10.0, 20.0);
211
212         double t = evoluir(u, u_new, versoes[v].fn, NT);
213         if (v == 0) t_base = t;
214         printf("%-28s  %-10.4f  %.4f\n",
215             versoes[v].nome, t, t_base / t);
216     }
217
218     free(u);
219     free(u_new);
220     free(base);
221     return 0;
222 }
223

```

```

1 import matplotlib
2 matplotlib.use("Agg") # noqa: E402
3 import matplotlib.pyplot as plt # noqa: E402
4 from matplotlib.animation import FuncAnimation # noqa: E402
5 from mpl_toolkits.mplot3d import Axes3D # noqa: E402, F401
6 import numpy as np # noqa: E402
7
8 NX, NY = 64, 64
9 R = 0.1 * 0.1 / 1.0 ** 2 # nu*dt/dx^2 = 0.01
10
11 cx, cy = NX // 2, NY // 2
12 x, y = np.mgrid[0:NX, 0:NY]
13 u = 10.0 * np.exp(-((x - cx) ** 2 + (y - cy) ** 2) / (2.0 * 8 ** 2))
14
15 X, Y = np.meshgrid(range(NY), range(NX))
16 passo = [0]
17
18
19 def step(u):
20     u_new = u.copy()
21     u_new[1:-1, 1:-1] = u[1:-1, 1:-1] + R * (
22         u[2:, 1:-1] + u[:-2, 1:-1] +
23         u[1:-1, 2:] + u[1:-1, :-2] - 4 * u[1:-1, 1:-1]
24     )
25     return u_new
26
27
28 fig = plt.figure()
29 ax = fig.add_subplot(111, projection="3d")
30
31
32 def update(frame):
33     global u
34     for _ in range(10):
35         u = step(u)
36         passo[0] += 1
37     ax.clear()
38     ax.plot_surface(X, Y, u, cmap="viridis", vmin=0, vmax=10)
39     ax.set_zlim(0, 10)
40     ax.set_xlabel("X")
41     ax.set_ylabel("Y")
42     ax.set_zlabel("Velocidade")
43     ax.set_title(f"Evolução da Difusão - Passo {passo[0]}")
44     return []
45
46
47 ani = FuncAnimation(fig, update, frames=100, interval=50)
48 ani.save("tarefa11/diffusion.gif", writer="pillow", fps=24)
49 print("Salvo: tarefa11/diffusion.gif")
50

```